

LLM-Driven Multi-Agent Framework for Autonomous Network Slice Management: Architecture and Proof-of-Concept Experiments

ABDULHALIM DANDOUSH¹ (Member, IEEE), VISWANATH KUMARSKANDPRIYA²,
MUEEN UDDIN¹ (Senior Member, IEEE), AHMED FAROUK³ (Member, IEEE), AND
MEROUANE DEBBAH⁴

¹College of Computing and Information Technology, University of Doha for Science and Technology (UDST), Doha, Qatar

²Computing, Modeling and Digital Technology Department, Esme Research Laboratory, SA ESME, Ivry-sur-Seine 94200, France

³Department of Computer Science, Faculty of Computers and Artificial Intelligence, Hurghada University, Hurghada 84516, Egypt

⁴Department of Computer and Information Engineering, KU 6G Research Center, Khalifa University, Abu Dhabi, United Arab Emirates

CORRESPONDING AUTHOR: A. DANDOUSH (abdulhalim.dandoush@udst.edu.qa)

ABSTRACT Network slicing (NS) enables service providers to create independent virtual networks, storage, and computing resources (so-called slices) from a shared physical infrastructure across one or more operators. These slices can be tailored to specific services, user needs, and Quality-of-Service (QoS) requirements. However, most production-grade orchestration frameworks deployed today still rely on manually crafted templates and centralized controllers, even though a growing body of research has begun to explore deep reinforcement learning (DRL) and multi-agent DRL (MARDL) for autonomous and distributed orchestration. As the number of slices increases, limited visibility across administrative domains and human-driven configuration render provisioning slow and error-prone. We address these limitations by proposing a multi-agent management framework where Large Language Models (LLMs) act as intelligent interfaces between tenants and the Management and Orchestration (MANO) stack. Agents interpret high-level intent provided by users or applications, translate it into standards-compliant descriptors, cooperate with other agents and underlying ML-based optimizers to allocate resources, and supervise the slice lifecycle. To validate this paradigm, we conduct two complementary experiments that together cover the AI-driven stages of the proposed workflow. First, we benchmark four state-of-the-art LLMs in generating Yet Another Markup Language (YAML)-based slice descriptions from natural-language prompts of varying complexity. GPT-4.1 consistently produces valid schemas and adheres to constraints, while other models occasionally violate conditions but still yield usable outputs. Second, we implement a proof-of-concept multi-agent system that, from a single user request, models a slice, synthesizes a Topology and Orchestration Specification for Cloud Applications (TOSCA) descriptor, and generates a deployment plan. This second experiment validates the multi-agent transformation chain from a slice request to deployable artifacts. The end-to-end workflow completes in about 40 seconds and produces artifacts validated by a domain expert. The final infrastructure instantiation of these already validated artifacts, e.g., through Kubernetes, Docker Compose, or NFV orchestration tools, is intentionally not treated as a separate AI experiment in this paper. Our results show that modern LLMs, when orchestrated in a collaborative agent framework, can significantly reduce the engineering effort and turnaround time required to move from high-level intents to deployable slice descriptors while ensuring alignment with industry standards. The proposed LLM-based framework can be integrated with any resource allocation optimizers or ML-based (e.g., DRL/MARDL) infrastructure and network controllers for multi-tenant resource optimization. We emphasize that the present validation targets the orchestration/specification layer of network slice management, namely the translation of natural-language intents into standards-compliant, internally consistent, and infrastructure-feasible artifacts. Open challenges and future research directions are discussed at the end of the paper.

INDEX TERMS LLM, NLP, network management, network orchestration, network slicing, multi-agent systems.

I. INTRODUCTION

THE ever-growing demand for diverse network services with varying Quality-of-Service (QoS) requirements has pushed the boundaries of traditional network architectures

[1]. NS emerges as a transformative technology addressing this challenge [2]. It allows the creation of isolated Virtual Networks (VNs) on top of a shared physical infrastructure. These slices can be customized with dedicated resources (connectivity, storage, computing) to meet the specific needs

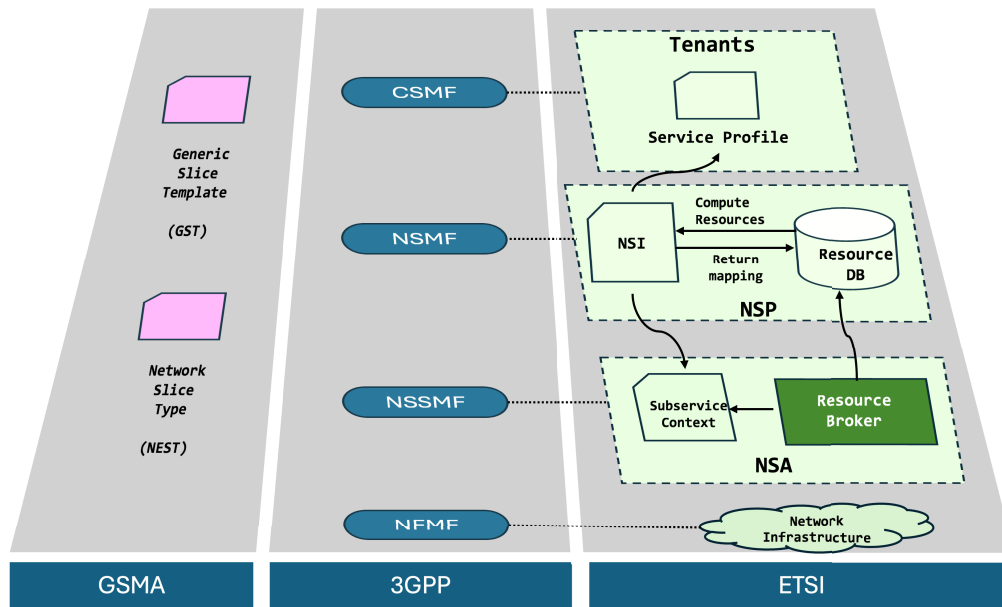


FIGURE 1. SDO's views on NS: GSMA, 3GPP and ETSI.

of a particular application or service, such as low-latency services for autonomous vehicles or high-bandwidth services for virtual reality experiences [3]. NS offers a multitude of advantages:

- **Flexibility and Agility:** Network slices can be provisioned, scaled, and reconfigured to meet dynamic service demands. This allows service providers to quickly adapt to changing market conditions and offer innovative services to their customers [1].
- **Enhanced Cost-efficiency:** Resources are efficiently utilized by dedicating them to specific applications, minimizing wastage. Network operators can optimize their infrastructure investments by allocating resources only where they are truly needed [2].
- **Multi-domain and Multi-tenant Support:** It empowers the creation of isolated network environments, catering to a variety of users and services across different administrative domains. This enables network operators to offer secure and reliable network slices to multiple tenants, such as enterprises and content providers [2].

In addition, NS has the potential to revolutionize various sectors, including:

- **Mobile Network Operators (MNOs):** Offering differentiated services with customized Service Level Agreements (SLAs) to diverse customer segments [4]. MNOs can leverage NS to create high-performance slices for mission-critical applications like industrial automation or remote surgery, while also providing basic connectivity slices for less demanding users (e.g., IoT application for smart parking).
- **Content Providers:** Delivering high-quality streaming experiences with guaranteed bandwidth and low latency. Content providers can create dedicated network slices to ensure smooth and uninterrupted delivery of video and audio content to their subscribers [5].

- **Enterprises:** Establishing secure and reliable private networks for mission-critical applications. Enterprises can leverage NS to create private slices for their internal applications, ensuring isolation, security, and predictable performance [6].

A. BASIC E2E NS ARCHITECTURE

The ETSI NS architecture (Figure 1) coordinates three principal actors, Tenants, NS Providers (NSPs), and NS Agents (NSAs), to provision isolated Virtual Network Instances (VNI) from shared physical infrastructure [2], [7]. The 3GPP vision, which is complementary, introduces three management functions: the Customer Service Management Function (CSMF), the NS Management Function (NSMF), and the NS Subnet Management Function (NSSMF). Together, these frameworks form the standards-compliant basis upon which our LLM-driven management layer is built, as detailed in Section III.

B. LIMITATIONS OF CURRENT ORCHESTRATION AND MANAGEMENT

While NS holds immense promise, current orchestration and management approaches primarily rely on SDN based solutions aligned with the specifications of centralized Network Functions Virtualisation (NFV); Management and Orchestration of ETSI [8], [9] or with a Multi-Level Delegation Architecture as described in [6]. Four critical limitations motivate this work:

- **Manual templates and centralized controllers** cannot scale to the diversity of 5G/6G service demands with heterogeneous QoS requirements [8].
- **Lack of dynamic negotiation:** Current user order specifications (like TMF's 461 [10] or MEF's 57 [11]) explain how orders should be managed but do not allow for dynamic negotiation, dramatically restricting flexibility [1].

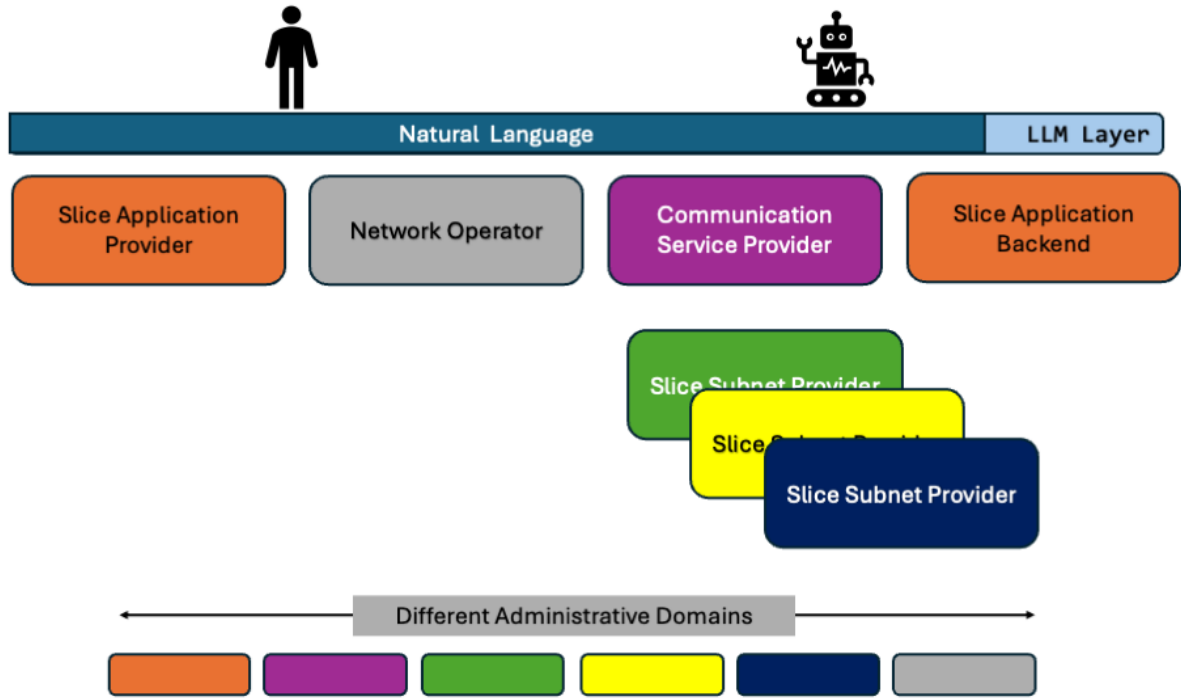


FIGURE 2. The position of a LLM layer in translating a user Intent spanning multi administrative domains.

- **Multi-domain complexity:** Each domain may have its own management system and protocols, making it difficult to achieve seamless slice creation across administrative boundaries [2], [6].
- **Slow provisioning:** Deploying a service without AI takes weeks or months as operators deal with hundreds of error-prone configuration commands [1]. In parallel, a rich research line has proposed DRL- and MARL-based orchestration schemes for specific slicing scenarios; however, these works typically assume structured slice specifications and do not address the end-to-end slice lifecycle management across multiple heterogeneous domains and incorporating the negotiation process relaxing certain user requirement when it is possible to increase slice admission rate.

Motivation: The Need for collaborative Intelligent Management The dynamic nature of network slices, coupled with the heterogeneity of services and multi-domain environments, necessitates not only an intelligent but also a collaborative management solution. A multi-agent system of Large Language Models (LLMs) can play a pivotal role in addressing challenges and issues in the telecom sector [12].

Consider a scenario where a network operator is performing network planning, or a non-technical user is subscribing to a paid service. In both cases, the user's intent is expressed in natural language, but these intents need to be translated into a structured language for different systems to work together. Additionally, multiple administrative domains must be considered when developing an end-to-end solution. Figure 2 illustrates this problem. Often, these domains use their own specific languages for design, deployment, and operations. Although standard organizations like 3GPP, ETSI,

TMForum, and IETF exist, specifications are frequently interpreted differently, creating significant interoperability gaps. This hinders time-to-market and agility in rolling out new services. LLMs can naturally fill this role by acting as interpreters between multiple domains. Figure 3 shows how different LLM agents can manage individual administrative domains while collaborating to achieve a common goal, without needing explicit programming. LLM agents can interpret user intents expressed in natural language and facilitate seamless communication between orchestrators across domains. They can also provide an abstracted view of the network topology within a region or autonomous system, allowing operators and slice providers to collaborate more effectively. These agents are distributed across the access, transport, core, and cloud networks. This enables the efficient creation or modification of slices spanning multiple domains. LLMs also contribute to adaptable resource allocation by dynamically analyzing patterns and determining optimal computing, memory, storage, and bandwidth resources. Furthermore, agents can map the chain of network functions required for a service to the shared infrastructure across different network domains. LLMs automate complex network configurations and instruction generation, ensuring accuracy and adherence to best practices. Through real-time monitoring and analysis, LLMs can report results to network controllers, allowing proactive measures to address issues and ensure uninterrupted performance.

At the same time, existing LLM-based configuration assistants are mostly constrained to single domains and device-level or controller-level configuration, while DRL/MARL-based network-slicing schemes excel at numerical resource optimisation but typically do not interpret natural-language intents or synthesize standards-compliant descriptors. This

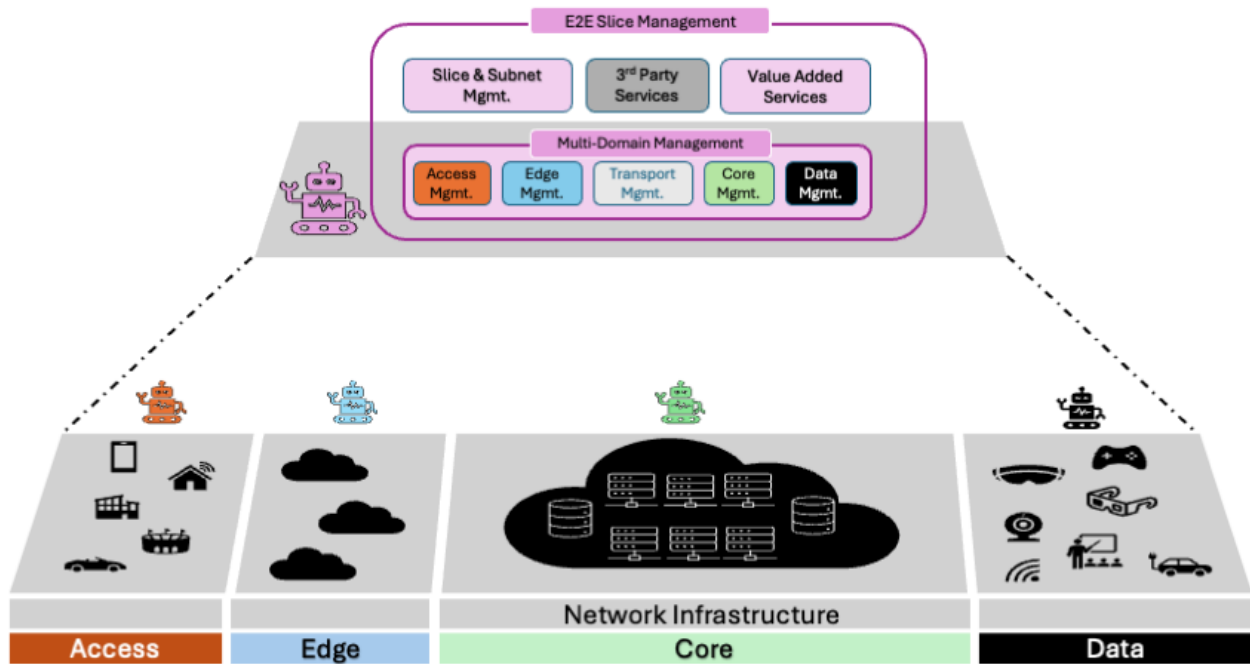


FIGURE 3. Collaboration between multi LLM agent from different administrative domain to achieve a common goal.

leaves a gap for an LLM-driven multi-agent layer that can (i) understand user intents, (ii) generate ETSI/3GPP/TOSCA-compliant slice artifacts, and (iii) coordinate across multiple administrative domains.

To the best of our knowledge, this is the first work in the literature to explore the potential of LLMs for E2E intelligent and autonomous management of Self-Driving Network Slices addressing the above tasks and challenges.

The main contributions of this paper are summarized as follows:

- **Multi-Agent LLM Architecture:** A novel framework mapping LLM-driven agents (e.g., Orchestrator, Modelling, and Deployment) to the ETSI/3GPP management stack, enabling autonomous slice lifecycle management (from natural-language intent to TOSCA-compliant deployment descriptors) across heterogeneous administrative domains with minimizing human intervention and substantially reducing the engineering effort and turnaround time required to move from high-level intents to deployable slice descriptors.
- **LLM Benchmarking for Slice Design:** We evaluate four state-of-the-art frontier LLMs on the task of translating natural-language intents into valid, standards-compliant YAML network slice configurations, using three complexity levels and four quantitative metrics (syntactic validity, schema adherence, referential integrity, and intent fulfillment).
- **End-to-End Proof-of-Concept:** We implement and validate the proposed framework via a working proof-of-concept (PoC) system, whose artifact workflows and generated results are verified by a domain expert. The PoC demonstrates that a complete intent-to-descriptor pipeline can be executed in about forty seconds, and we discuss how this LLM-based specification layer can

be integrated with DRL/MARL-based controllers and extended to multi-tenant resource contention scenarios.

To avoid overstating the scope of the paper, we stress that the proposed contribution is not a replacement for heuristic, optimization-based, or DRL/MARL-based controllers. Rather, it is a complementary LLM-driven orchestration/specification layer that sits above such engines, translates human intents into standards-compliant artifacts, and coordinates multi-domain slice lifecycle actions before downstream resource-allocation mechanisms enforce network-level KPIs.

For convenience, Table 1 provides a glossary of the main acronyms used throughout this paper.

The rest of this paper is organized as follows: A comparative overview of the current application of ML for NS is presented in Section II. In Section III the multi-agent LLMs based approach is described from both the user and operator perspectives and in Section IV, we discuss our experimentation details and results. The challenges of applying LLMs for network management and orchestration are discussed in Section V. Section VI concludes our work. Future research directions are outlined in Section VII. Last we provide some technical details in the Appendix A and B and through shared GitHub link.

II. RELATED WORKS

Generative AI and large language models (LLMs) have recently been proposed as building blocks for automating many network-operations tasks. The ability of an LLM to parse natural-language requests and synthesize technical configurations makes it a promising assistant for report generation, data analysis, network automation, outage handling and business-process optimization. Nevertheless, LLMs must be integrated into network management frameworks that respect industry standards, support multi-domain coordination and

TABLE 1. List of key acronyms.

Acronym	Definition
3GPP	3rd Generation Partnership Project
A2A	Agent-to-Agent
ETSI	European Telecom Standards Institute
GST	Generic Slice Template
IBN	Intent-Based Networking
LLM	Large Language Model
MANO	Management and Orchestration
MNO	Mobile Network Operator
NOP	Network Operator
NFV	Net. Functions Virtualisation
NSD	Net. Service Descriptor
NSMF	Net. Slice Management Function
CSMF	Communication Service Mgt Function
NSP	Net. Slice Provider
NSSMF	Net. Slice Subnet Management Function
NS	Network Slicing
NSI	Network Slice Instance
PoC	Proof-of-Concept
QoS	Quality of Service
SDN	Software-Defined Networking
SLA	Service Level Agreement
VIM	Virtual Infrastructure Manager
VN	Virtual Network

TABLE 2. Comparative summary of related works.

Work	Approach	Multi-Domain	Intent	Standard	PoC
[13]	LLM + SDN	No	Partial	No	Yes
[14]	Local LLM	No	Yes	No	Yes
[15]	In-context LLM	No	Yes	No	Sim.
[21]	MARL	Yes	No	No	Sim.
[22]	LLM Agent	No	Yes	No	Sim.
[19]	NLP + 5G	No	Yes	Partial	Yes
Ours	Multi-agent LLM	Yes	Yes	Yes	Yes

provide trustworthy operation. This section organizes the most relevant literature by theme and discusses how our proposed multi-agent LLM framework advances the state of the art.

A. LLMs FOR NETWORK CONFIGURATION AND MANAGEMENT

Early studies explored how LLMs could simplify network configuration by translating high-level policies into low-level device commands. Wang et al. proposed in [13] *NetConfEval*, a set of benchmarks that use GPT-4 to generate low-level

network configurations and Python code (e.g., P4) from high-level goals and employs a self-healing procedure to correct errors at runtime. The *LLM_NetCFG* framework by Lira et al. takes a complementary approach by running a Zephyr-7B model locally to translate intents into validated device configurations, thereby protecting operator privacy and reducing reliance on remote services [14]. While *LLM_NetCFG* shows that local LLMs can automate configuration generation and verification, its scope remains confined to device-level configuration within one administrative domain.

Beyond intent-configuration translation, researchers have begun to develop LLMs tailored to future intent-based networks. Van et al. design a custom LLM for next-generation 5G/6G networks and argue that intent-based management will be key to achieving fully automated network intelligence [15]. Zhou et al. introduce *NFV-Intent*, which employs in-context learning to map natural-language intents into Network Function Virtualization (NFV) configurations without extensive retraining of the underlying LLM [16]. These systems primarily target NFV life-cycle management and do not extend to end-to-end (E2E) slice orchestration or multi-domain operation. Recent surveys highlight that most LLM-assisted configuration tools focus on small networks, often rely on handcrafted datasets and still suffer from hallucination and scalability issues [17], [18]. Our work moves beyond device-level configuration by leveraging cooperating LLM agents to manage the entire network slice lifecycle across multiple domains.

B. INTENT-BASED NETWORKING AND HIERARCHICAL NETWORK SERVICES

Intent-based networking (IBN) aims to map user goals into concrete network actions. McNamara et al. [19] present an intent-driven platform for private 5G networks powered by a natural-language interface, demonstrating use-cases such as slice provisioning and service deployment. Their implementation clarifies how an NLP front-end can simplify operator tasks, but it operates within a single administrative domain under one SDN controller or cluster. Tarkoma et al. also build an NLP-enabled platform for intent-driven networking with case studies on slice provisioning and service deployment [20]. These works illustrate the promise of IBN but do not address the challenges of multi-domain interoperability or standard compliance.

Related to IBN is the concept of hierarchical network slicing as a service (NSaaS). McNamara et al. describe how operators can offer customised E2E cellular networks to vertical industries by composing slices through hierarchical service orchestrations and aligning service-level agreements across layers [19]. However, they do not explore how AI could automate these processes. Our architecture builds upon the IBN paradigm by introducing multiple LLM agents that interpret high-level intents, generate standards-compliant descriptors and coordinate with underlying management and orchestration systems across domains.

C. MULTI-AGENT SYSTEMS AND RESOURCE-OPTIMISATION APPROACHES

Existing work on multi-agent systems for network slicing focuses mainly on numerical optimization and resource allocation. Mao proposes in [21] a cooperative multi-agent

reinforcement-learning (MARL) framework in which each slice tenant acts as an agent learning resource-sharing strategies; hierarchical attention and credit-based evaluation are used to improve learning efficiency and achieve higher utility and SLA satisfaction than baseline methods. While effective for resource optimization of MEC enabled 5G computing resources, such MARL approaches do not handle natural-language intent interpretation or standards-based configuration synthesis for the E2E slice lifecycle management. Our framework differs by using LLMs as intelligent agents capable of understanding human-expressed intents and generating slice descriptors in accordance with ETSI and 3GPP specifications and translate them into low level network configuration while monitor and adapt resources in real-time according to the service need.

LLM-centric agent frameworks have also begun to appear. Tong et al. propose *WirelessAgent*, an LLM-powered agent system with modules for perception, planning and action; their case studies show accurate intent understanding and near-optimal slice resource allocation in wireless networks [22]. However, *WirelessAgent* does not address standardized orchestration or multi-domain coordination. Our work is narrower in focus but deeper in scope. We design a multi-agent system specifically for E2E network slice management across multiple administrative domains from user request to the services hosted in some server (e.g. Cloud) crossing Teleco and network operators. The orchestrator, modelling and deployment agents interpret a user's natural-language intent, produce standards-compliant slice models (i.e., ETSI/3GPP) and TOSCA network-service descriptors, and execute deployment plans. A proof-of-concept implementation shows that our pipeline can translate an intent into validated artifacts in about tens of seconds compressing a process that normally takes days or weeks.

Several surveys and white papers provide broader overviews. The 5G PPP white paper surveys various neural-network techniques for planning, diagnostics and optimization in 5G and 6G networks, noting that most solutions target specific slices or Virtual functions config or placement rather than the full lifecycle [23]. Surveys by Hong et al. [17] and Liu et al. [18] systematically review LLM-based network management methods, emphasizing challenges such as hallucinations, domain adaptation and the scarcity of standardized datasets. These reviews conclude that existing efforts rarely consider multi-domain environments and that integrating LLMs with network management frameworks remains an open problem. Our work addresses this gap by proposing a multi-agent architecture in which LLM agents collaborate to interpret intents, generate slice descriptors, allocate resources and supervise deployments across multiple administrative domains. Crucially, the proposed LLM-based specification layer is orthogonal to any Machine Learning based (e.g., DRL/MARL) resource-allocation engines. In our example, the generated standards-compliant descriptors can serve as structured inputs to any downstream optimizer, and a ML (like DRL/MARL) controller could in turn feed back KPI data to guide the agents at runtime.

D. SYNTHESIS AND RESEARCH GAP

The survey of related work reveals three converging insights. First, while LLMs have been demonstrated for device-level

configuration (single-domain, single-controller), no prior work addresses the full E2E slice lifecycle across multiple administrative domains with standards-compliant descriptors. Second, multi-agent RL approaches optimize resource allocation effectively but cannot interpret natural-language intents or generate standards-based configuration artifacts. Third, intent-based networking platforms exist but operate within single administrative domains and without AI-driven automation of descriptor synthesis or deployment. Our framework uniquely bridges these gaps by combining LLM-driven intent interpretation, standards-compliant artifact generation (ETSI/3GPP/TOSCA), multi-domain coordination via agent-to-agent APIs, and an experimentally validated PoC. Accordingly, the main comparative lens of this paper is the specification/orchestration layer—intent understanding, standards compliance, artifact consistency, and workflow automation—rather than controller-level KPI optimization, which belongs to a different and complementary problem setting.

The agents in our framework are powered by decoder-only Large Language Models (LLMs); generative models that produce text autoregressively, conditioned on a context window [24]. A key property exploited in our design is *in-context learning*: by including structured examples, infrastructure descriptions, and tool definitions in the system prompt, LLMs can perform complex reasoning and generate standards-compliant artifacts without fine-tuning on domain-specific data [25].

III. PROPOSED LLM-BASED SLICING MANAGEMENT APPROACH

In this section, we describe our proposed approach for utilizing LLM based Multi Agent Systems as a transformative solution for NS management. Figure 4 describes how the set of LLM agents can be mapped to management frameworks introduced by SDOs such as ETSI and 3GPP. Each agent within the system represents a distinct administrative domain, or alternatively, a Software-Defined Networking (SDN) controller responsible for specific segment subnet configurations and data/log/statistic collection from the underlying infrastructure. Notably, an agent can also function as an SDN orchestrator, synthesizing a high-level abstraction of network, storage, and computing resources obtained from the agents working at the SDN controllers level in each domain. This orchestrator agents plays a pivotal role in exposing and communicating an abstracted view of the topology and the available capacity to other agents (i.e., on the other administrative domains). Through this cooperative and intelligent multi-agent system, we envision the collaborative management of end-to-end NS that overcomes the challenges and limitations posed by the deployment of the SDOs NS frameworks in multi-administrative domains with highly heterogeneous services as discussed in Section V.

The general position of the LLMs multi-agent in E2E NS Management and Orchestration is illustrated in Figure 5.

LLM Assisted functions can seamlessly collaborate and coordinate with each others thereby offering enriched user experience to both categories of users: (i) Network Operators and (ii) Network Consumers. In following subsections, we will explain the improved user experience flows for each category of users.

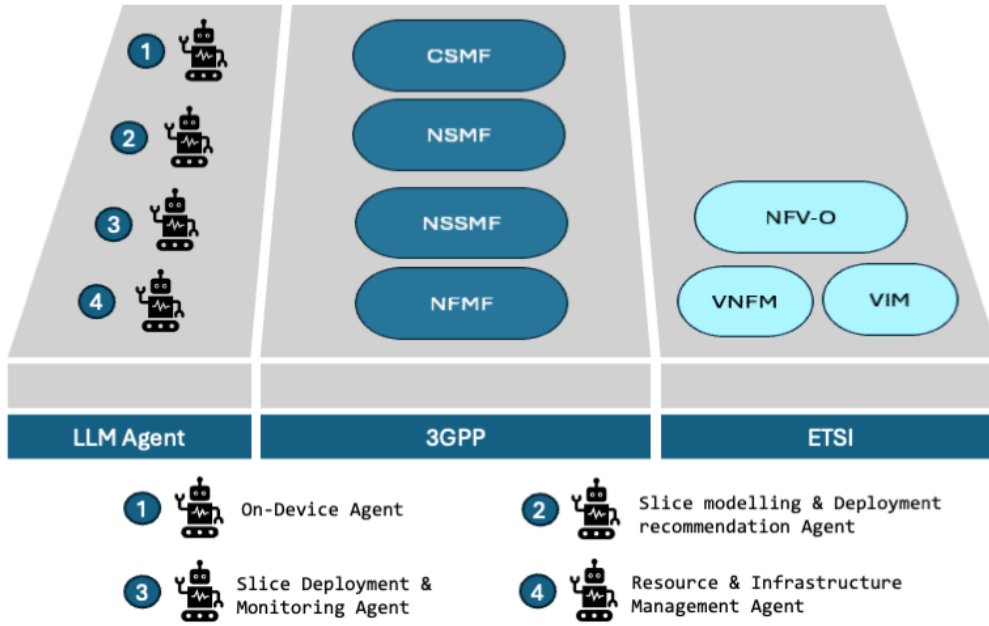


FIGURE 4. LLM Agents scope mapped to 3GPP and ETSI related management functions responsible for network slice deployments.

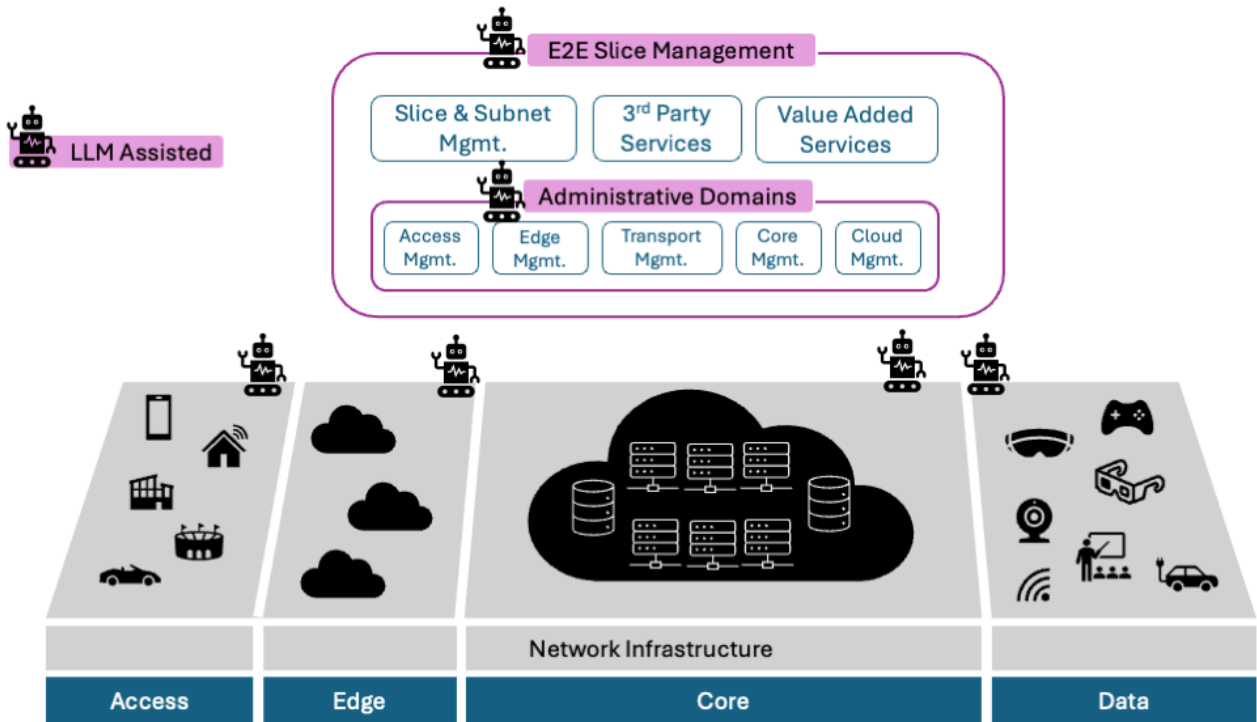


FIGURE 5. High level view of LLMs assisted network slice management.

A. FORMAL PROBLEM FORMULATION

Let the network infrastructure be represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of physical nodes (cloud, edge, access) and $\mathcal{E}$ is the set of links. Each node $v \in \mathcal{V}$ has a resource vector $\mathbf{r}_v = (r_v^{\text{cpu}}, r_v^{\text{ram}}, r_v^{\text{hdd}})$, and each link $e \in \mathcal{E}$ has

a bandwidth capacity b_e . We denote by $\mathcal{S}$ the set of all active network slices in the system.

A *network slice request* from a tenant is represented as a natural-language intent I (a text string). The Orchestrator Agent $\mathcal{A}_O$ applies an LLM-based function f_θ to map I to a

structured technical specification S :

$$S = f_{\theta}(I, \mathcal{K}), \quad (1)$$

where $\mathcal{K}$ represents the injected domain knowledge context (infrastructure description, standards templates, tool definitions), and θ denotes the LLM parameters.

The specification S encodes the service-level requirements of the requested slice, including QoS targets such as maximum end-to-end latency $L_s^{\max}$, minimum throughput $T_s^{\min}$, and target reliability $R_s^{\min}$ for slice s . The Modelling Agent $\mathcal{A}_M$ then maps S to a slice topology $T_s = (V_s, E_s, \mathbf{R}_s)$, where $V_s \subseteq \mathcal{V}$, $E_s \subseteq \mathcal{E}$, and $\mathbf{R}_s$ specifies the resource allocation for each node in the slice.

The allocation must satisfy node capacity constraints across *all* active slices:

$$\sum_{s \in \mathcal{S}} r_{s,v}^k \leq r_v^k, \quad \forall v \in \mathcal{V}, \quad \forall k \in \{\text{cpu, ram, hdd}\}, \quad (2)$$

where $r_{s,v}^k$ is the resource of type k allocated to slice s on node v . Similarly, link capacities must not be exceeded across all active slices:

$$\sum_{s \in \mathcal{S}} b_{s,e} \leq b_e, \quad \forall e \in \mathcal{E}, \quad (3)$$

where $b_{s,e}$ is the bandwidth reserved for slice s on link e .

QoS requirements derived from S impose additional constraints on feasible topologies. The end-to-end latency L_s of slice s must satisfy

$$L_s \leq L_s^{\max}, \quad (4)$$

the achievable throughput T_s must satisfy

$$T_s \geq T_s^{\min}, \quad (5)$$

and the reliability must satisfy $R_s \geq R_s^{\min}$. These constraints ensure that the generated slice descriptors remain semantically valid with respect to the physical infrastructure, directly limiting a class of LLM semantic hallucinations where the model would otherwise request infeasible resources or violate latency bounds.

The optimization objective for the Modelling Agent is to find a feasible T_s that maximizes the average slice utilization ratio ρ :

$$\max_{\{T_s\}_{s \in \mathcal{S}}} \rho = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \frac{\sum_{v \in V_s} \sum_k r_{s,v}^k}{\sum_{v \in V_s} \sum_k r_v^k}, \quad (6)$$

subject to constraints (2)–(5) and the reliability constraint $R_s \geq R_s^{\min}$. Other objectives, such as fairness or energy efficiency, could also be incorporated at the resource-manager layer.

Finally, the Deployment Agent $\mathcal{A}_D$ translates T_s into a deployable Network Service Descriptor:

$$\text{NSD}_s = g_{\theta}(T_s, \mathcal{K}_{\text{TOSCA}}), \quad (7)$$

where g_{θ} is the LLM-based generation function and $\mathcal{K}_{\text{TOSCA}}$ is the TOSCA/ETSI schema injected as context. In our proof-of-concept, every candidate descriptor generated by the agents is subsequently checked against constraints (2)–(5), ensuring that LLM-generated artifacts remain feasible with respect to the physical infrastructure and advertised QoS requirements before being deemed deployable.

B. FROM CUSTOMER REQUEST TO AUTO DEPLOYMENT

1) SERVICE CONSUMER POINT OF VIEW

LLMs play a crucial role in simplifying the interaction between end users or applications and the network orchestration system. When a user initiates a network slice request, she/he often communicates her/his needs in natural language, specifying high-level requirements such as good connectivity, low cost or premium service, high or low “connection speed”, and reliability. A LLM model as shown in Figure 6 can translate this high level request to a more concrete service profile, identifying more technical requirements such as latency, throughput, and reliability according to the slice template request standardized by GSMA/3GPP. In other words, LLMs excel in translating these user requests into specific technical requirements that can be understood and executed by the network orchestrator(s). Moreover, if the required constraints cannot be met due to capacity limitation on some portion of the slice, the agent can negotiate (e.g., by sending a specific prompt) with the corresponding agent located at the access network level to negotiate a relaxed version of the requirements that can be allocated and initiated if the user can approve them. As discussed in the introduction, the current user order specifications (like TMF’s 461 [10]) do not allow for dynamic negotiation, which dramatically restricts the flexibility of their approach.

Leveraging their natural language understanding capabilities, LLMs ensure precise communication between users and the network slice provider related agents. Figure 6 shows a hypothetical scenario where an end user is requesting one or two slices for Telemedicine service. In fact, based on the scenario, a telemedicine service can consume two slices, one with high throughput for the remote visio and video streaming and another with an ultra low latency requirement used to send commands remotely to a medical devices (e.g., active catheter or robots). The user simply states their intent of consuming a Telemedicine service along with the requirement for high quality video calling and security, considering the privacy and safety concerns related to the health data.

A chatbot agent can assist the user at the early stage to form any request. The agent then can prompt the corresponding agent at the NS provider to suggest a new chain of functions based on its knowledge and continuous learning of different patterns. The user usually has zero knowledge how the underlying network infrastructure is being built and what network slices are available or need to be created in-order to deliver the Telemedicine or another innovative service. Here a LLM agent that co-reside on the user equipment (a PC or a medical device) can quickly provide on-device NLP experience. The LLM agent can synthesize and understand the user’s intent and efficiently translate that to a 3GPP service profile in terms of standard service attributes that can be exchanged with a standards compliant Communication Service Management Function (CSMF) and Network Slice Management Function (NSMF) respectively.

Here in this case, multiple LLM agents can work at different administrative domain to execute a control task. For example LLM agents at NSMF can coordinate with LLM agents as Network Slice Subnet Management functions (NSSMF) such as Core, Access and Transport subdomains to effectively configure underlying network infrastructure as per the network intents specified in the Slice profile. In fact,

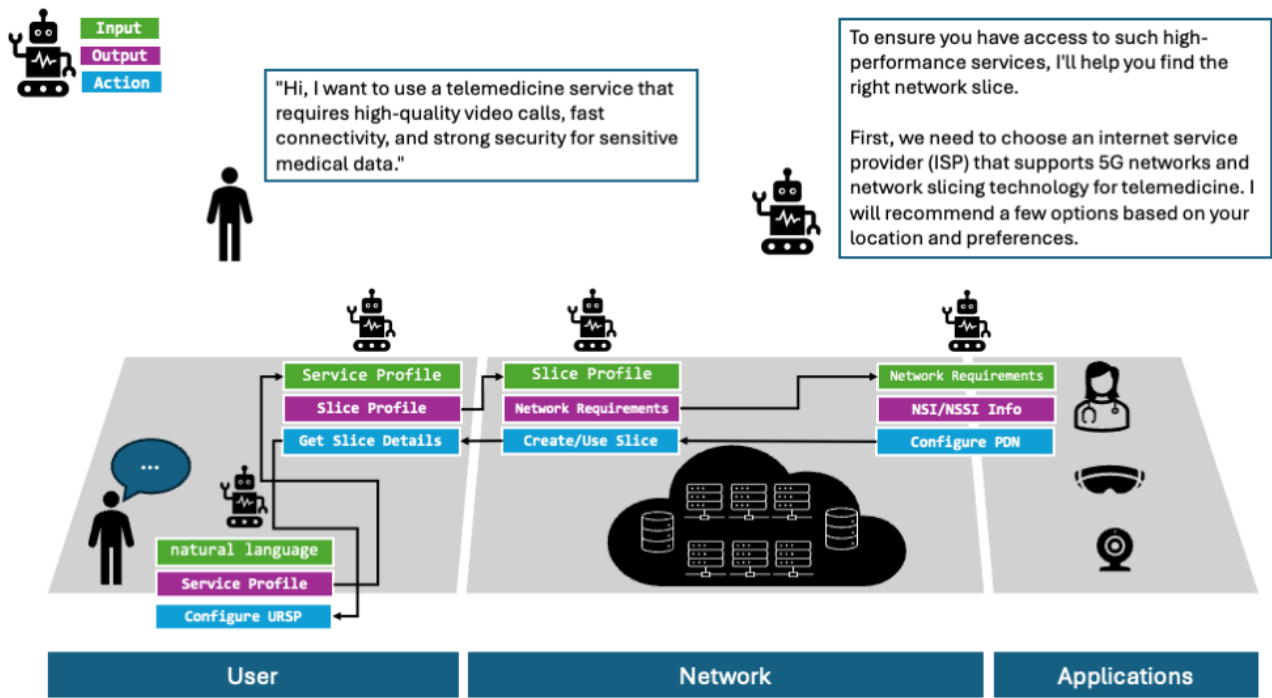


FIGURE 6. LLM Assisted End User Network Slice Intent Translation.

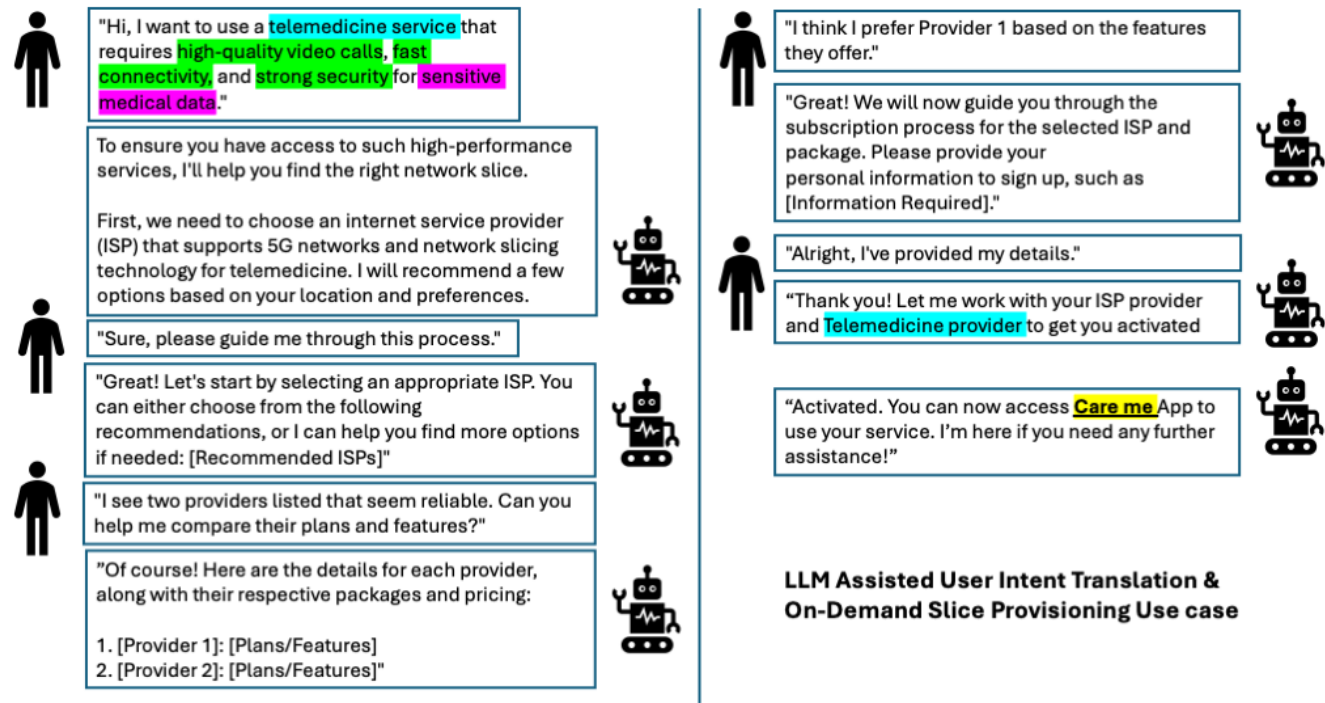


FIGURE 7. A hypothetical interaction between an end user and an intelligent LLM, negotiating over a telemedicine service.

intelligent mapping of one or more network slice profiles to a service profile can be efficiently performed a utility LLM agent that is sitting between NSMF and NSSMFs respectively.

Introducing LLM assisted capability in service fulfillment will not only improve automation and time to efficiency, but also will vastly improve end user experience. Figure 7 depicts

a hypothetical natural language conversation between an end user and the on-device LLM agent.

LLM agent first extracts the keywords from the user's intent (such as telemedicine service, high quality video calls, security, sensitive medical data) and deduces that for this service to be provided, a special network slice has to be utilized. LLM Agent also deduces that a 5G and 6G NR

Agent	Goal	Depends On (tools)		
		Tools	Input	Output
On-Device Agent 	Design and Deploy a network slice infrastructure within the constraints given by the network operator	Slice modelling Agent	network slice modelling requirements	slice topology recommendations
		Slice Deployment Agent	network slice deployment information	deployed network slice information
Slice Modelling Agent 	Model a network slice according to the given constraint by performing experiments on various network topologies to arrive at recommendations	slicenet [18]	*yaml files representing slicenet experiments	slice topology recommendations and Slice Profiles for each recommendation
		slicenet-preparer	network slice modelling requirements	*yaml files representing slicenet experiments
Slice Deployment Agent 	Deploy a network slice topology using a given network infrastructure by using standards based network management frameworks	nsi-deployer	Slice deployment descriptors	Network Slice Instance Information
		nsi-preparer	network slice profile based on GSMA NEST	TOSCA YAML files in ETSI SOL 007 NSD and ETSI SOL 001 VNFD or similar deployment descriptors
Resource Management Agent 	Manage Virtual or Physical network infrastructure resources and serve resource allocation request at the VIM layer using standards based network management frameworks	resource-deployer	NF deployment descriptors	Network Function Instance Information
		resource-manager	network function instance information & resource management actions {start, stop, scale, restart, delete}	NF Monitoring information

FIGURE 8. Typical examples of finetuned LLM Agents.

access or a fiber optical broadband access is very crucial for the ultra low latency service requirement.

2) NETWORK OPERATOR POINT OF VIEW

The potential for LLMs to assist Network Operators in planning and deployment of network slices is enormous. In this section, we explore some of the possible use-cases. As in the case of network service consumer, a network operator user can express their intent of deploying one or more network slices along with specific optimization goals for each of the network slice. This intent along with information about the underlying network infrastructure (such as access, transport, core and datacenter network pieces of information) can be used by series of LLM agents to coordinate a network modelling exercise.

Figure 8 describes the composition of multi-agent framework for LLM assisted network slice design and deployment, which can be used by a network operator. Each agent is created with a defined role, goal, inputs to consume and outputs to produce. Also each agent is equipped with a set of tools to carry out their respective tasks. The *On-Device Agent* is the higher-level user facing agent that resides within the user equipment to facilitate the user interaction with the Network Slice Provider (NSP) or the Network Operator (NOP). This agent has access to *Slice modelling Agent* and *Slice Deployment Agent* as part of *tools* declaration within the agent's definition. Each tool has its own *inputs* and *outputs* respectively. LLM learns what input to pass and what output to collect based on the respective tool's definition. A complete sample listing of network slice LLM agent definition is provided in the appendix A section as a Yaml template file, where Yaml is a human-

friendly data serialization language for all programming languages.

The routing of tasks among agents follows a *tool-calling* paradigm. Each specialized agent (i.e., an Agent trained on a specific role like Modelling Agent, Deployment Agent, and Resource Management Agent) is registered as a callable "tool" within the Orchestrator Agent's configuration (see the YAML definition in Appendix A). When the Orchestrator receives a user intent, its underlying LLM performs chain-of-thought reasoning to decompose the goal into an ordered sequence of sub-tasks. For each sub-task, the Orchestrator selects the appropriate agent-tool by matching the task description against the natural-language *description* field of each declared tool. It then generates the required input payload and dispatches an Agent-to-Agent (A2A) API call to the selected agent. The called agent processes its input, executes any internal tool calls (e.g., invoking the SliceNet simulator or the NSD generator), and returns a structured JSON response. This response is ingested by the Orchestrator as new context for planning the next sub-task. The routing is therefore entirely governed by the LLM's in-context reasoning (no hard-coded rule-based router is needed) making the framework naturally extensible to new agent types.

Regarding agent knowledge and infrastructure awareness, a key architectural principle is that domain-specific agents (i.e., agents trained for a specific role) are self-reliant: the Orchestrator (caller) conveys only the service intent and request metadata, such as resource requirements, SLA targets, and domain scope, while the callee agent independently fulfills the request using its own registered tools and domain knowledge. Each agent maintains two agent-local information sources. First, *static domain knowledge* is

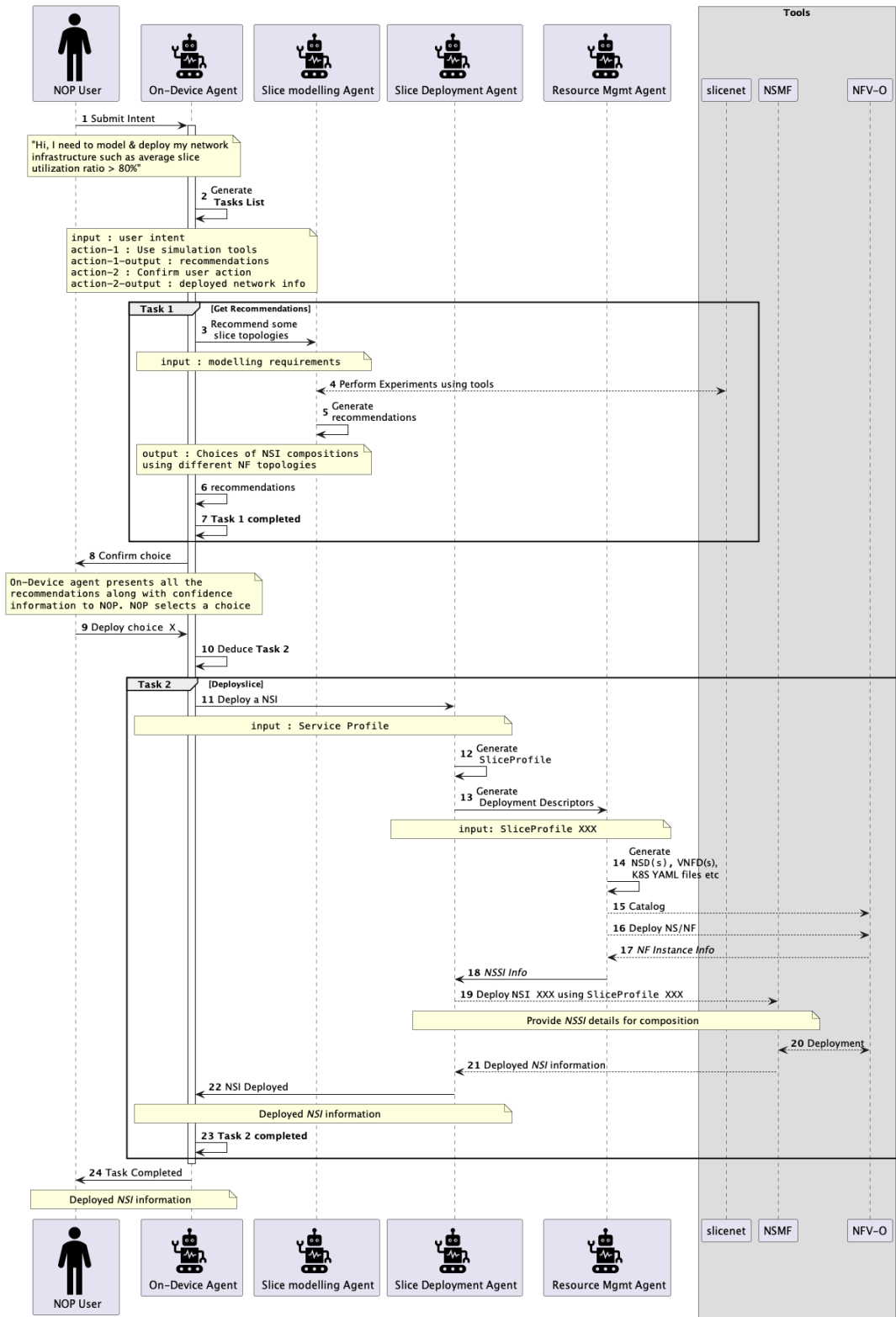


FIGURE 9. Sequence of interaction among Slice agents to design and deploy a network slice against a specific optimization constraint.

injected into the agent’s system prompt at instantiation: the network operator provides a structured YAML description of the infrastructure within that agent’s domain (cloud nodes, network function types, resource capacities like compute, storage, bandwidth, and inter-domain connectivity) which the

agent uses as its ground-truth reference for constraint checking and planning. Second, *dynamic resolution* is performed autonomously via agent-owned tools, though the nature of this self-reliance differs by agent type. Infrastructure-facing agents, such as the *Resource Management and Deploy-*

ment agents, invoke a domain-scoped resource-manager tool (Appendix A) registered directly with them to query the Virtual Infrastructure Manager (VIM) or NFV Orchestrator northbound API (e.g., ETSI SOL005/SOL002), returning up-to-date utilization and availability metrics as a structured JSON object. The Modelling Agent, by contrast, does not query live infrastructure; instead, its self-reliance lies in its own registered analytical and simulation tools, which it uses to perform topology design and optimization purely over the intent, constraints, and resource metadata passed by the Orchestrator. This design ensures clean separation of concerns. The Orchestrator reasons over intent and coordination, while each domain agent encapsulates all domain-specific resolution logic appropriate to its role decoupling the LLM reasoning layer from any particular management system and ensuring agents always operate over updated, accurate infrastructure data within their respective domains.

Figure 9 explains how these LLM agents can work collaboratively for a network operator use-case. Consider a scenario where a network operator would like to design and deploy a network slice instance within an underlying network infrastructure with limited resources such that, the average slice utilization ratio is greater than 80 percent (for optimizing power and cost). Without LLM assistance, this process would involve considerable effort as a network planning activity involving a long journey [1]. However this entire experience can be rapidly automated with LLMs in picture.

In this typical use-case, the network operator simply expresses its intent to the *On-Device* agent which is capable of using *Slice modelling* and *Slice Deployment Agents*. First it deduces the tasks to be followed. In this case, it learns first the slice modelling has to be done to generate some recommendations on different slice topologies that would fit the optimization constraint of maintaining overall slice utilization ratio of greater than 80 percent. In-order to perform this task, it chooses the *Slice modelling Agent* based on the description. Since this tool requires some input, *On-Device agent* generates that input based on its knowledge and call the *Slice modelling Agent* to perform Task 1 as shown in Figure 9. *Slice modelling Agent* in-turn uses another tool like slicenet [26] to perform the necessary experiments and modelling of the network to fit the optimization criteria. Based on the insights, *Slice modelling agent* further generates some recommendations on a different Network Slice Instance (NSI) topologies that could fit the optimization goal and pass this as the output to *On-Device agent* (refer to step #7 in sequence diagram depicted in Figure 9). This is further presented back to the NOP user to confirm its choice (as shown in step#8 in the figure). Here the user has option to either accept one of the choice provided by the LLM or provide reinforcement learning from human/AI (RLHF) feedback to fine tune the modelling task as needed. If the NOP user proceeds to select a choice, *On-Device agent* proceeds to Task 2. In this task, *On-Device agent* works with *Slice Deployment and Resource Management Agents* respectively. These lower level agents are capable of generating standards based deployment descriptors such as *SliceProfile* [27], *Network Service Descriptor* [28], *Network Function Descriptor* [29] and also interact with Network Slice Management Systems, NFV-Os to deploy the

network slice instances. Post deployment, the information about the created NSI(s) along with other site specific information is relayed back to the *On-Device agent* thereby concluding Task 2.

Last, concerning the deployment of Task2, by LLMs assisting in the generation of specific requirements, such as computing, memory, storage, and bandwidth resources across each administrative domain, the NSP ensures that the orchestrators at different domains are equipped with accurate and standardized information templates, promoting consistency and interoperability (e.g., using TOSCA Yaml as shown in Appendix A). Moreover, LLMs support the mapping of user requirements to network functions and services. This involves analyzing the placement of functions, which is a critical step in optimizing resource allocation and then converting the map into actionable configurations, promoting a harmonized approach to NS management. Also, one of the critical challenges in NS is the optimal allocation of resources to different slices, as formalized in Eq. (6). LLMs, with their ability to analyze and understand complex patterns and requirements, can determine dynamically the appropriate computing, memory, storage, and bandwidth resources (e.g., auto-scaling up or down) at a given time based on the collected statistics from the southbound interfaces such as P4 or OpenFlow, resulting in a more efficient network functioning.

IV. EXPERIMENTAL VALIDATION

This section presents the experimental validation of our proposed framework. We designed two complementary experiments to (1) evaluate the capability of modern LLMs in generating valid network slice configurations and (2) demonstrate the effectiveness of our multi-agent approach in a practical proof-of-concept (PoC). Experiment 1 validates the upstream intent-to-slice-description stage, whereas Experiment 2 validates the downstream multi-agent orchestration stage that transforms a slice request into a logical model, a standards-compliant TOSCA descriptor, and a deployment plan. In terms of the workflow illustrated in Figure 13, the reported experiments cover the AI-driven interactions up to the point where validated deployment artifacts are produced. The remaining interactions correspond to the actual instantiation of these artifacts by the infrastructure layer. For this reason, the final infrastructure instantiation is intentionally not evaluated as a separate AI experiment because, once the YAML/TOSCA outputs are generated and checked, their execution is handled by existing infrastructure managers such as Kubernetes, Docker Compose, or NFV orchestration tools.

A. EXPERIMENT 1: BENCHMARKING LLM CONFIGURATION GENERATION

The objective of this experiment was to assess the proficiency of various frontier LLMs in translating natural language user intent into structured, machine-readable network slice configurations. We designed three prompts of increasing complexity (full prompts text is available here [30]):

- 1) **Prompt 1 (Simple and Explicit):** A prompt that tests the LLM's basic ability to follow direct instructions and generate a syntactically correct YAML file with minimal inference required.
- 2) **Prompt 2 (Intermediate):** A prompt that is more advanced and requires the LLM to infer technical details from a high level, natural language request.

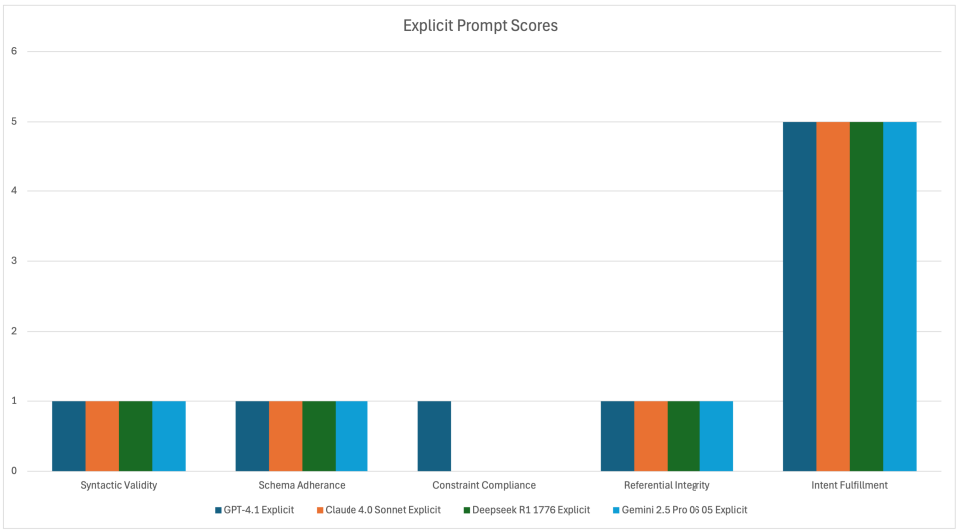


FIGURE 10. Performance of the large language models for the simple explicit prompt. Four frontier LLMs were evaluated against the standard metrics for syntactic validity, schema adherence, referential integrity and intent fulfillment.

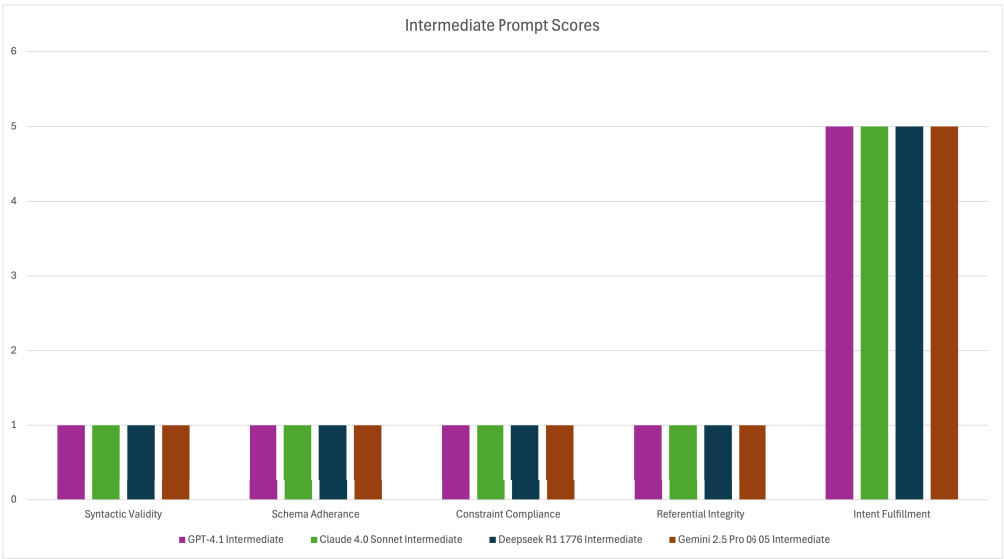


FIGURE 11. Performance of the large language models for the intermediate prompt.

- It specifically tests the LLM’s ability to adhere to a negative constraint (omitting a key when a condition is met), which is a core challenge in intent translation.
- 3) **Prompt 3 (Advanced):** A prompt that tests the LLM’s ability to manage a complex structure with many interrelated components. It assesses its capacity to maintain referential integrity across multiple lists (nfs, slices, services, slicelets) and to correctly interpret a multi-faceted request, which is representative of a real-world network planning scenario.
- To quantitatively and qualitatively assess the quality of the generated YAML configurations, we defined four key metrics:
- **Syntactic Validity:** Measures whether the generated output is a valid YAML file that can be successfully parsed. This is measured as a binary score (0 or 1).

- **Schema Adherence:** Evaluates if the YAML structure and data types conform to a predefined schema. For this experiment, the slicenet experiment configuration schema was used. This is scored on a scale from 0 to 1 based on the percentage of compliant fields.
- **Referential Integrity:** Checks if all internal References within the configuration are valid (e.g., a network function reference in a slice composition really exists and a slice reference in a service composition really exists and so on). This is scored based on the percentage of valid references, represented in floating point scale from 0 to 1.
- **Intent Fulfillment:** A qualitative score (from 1 to 5, where 5 being best) assigned by a human expert, assessing how well the configuration captures the intent of the original natural language prompt. This score was also valid using Grok 3 Model acting as LLM as Judge

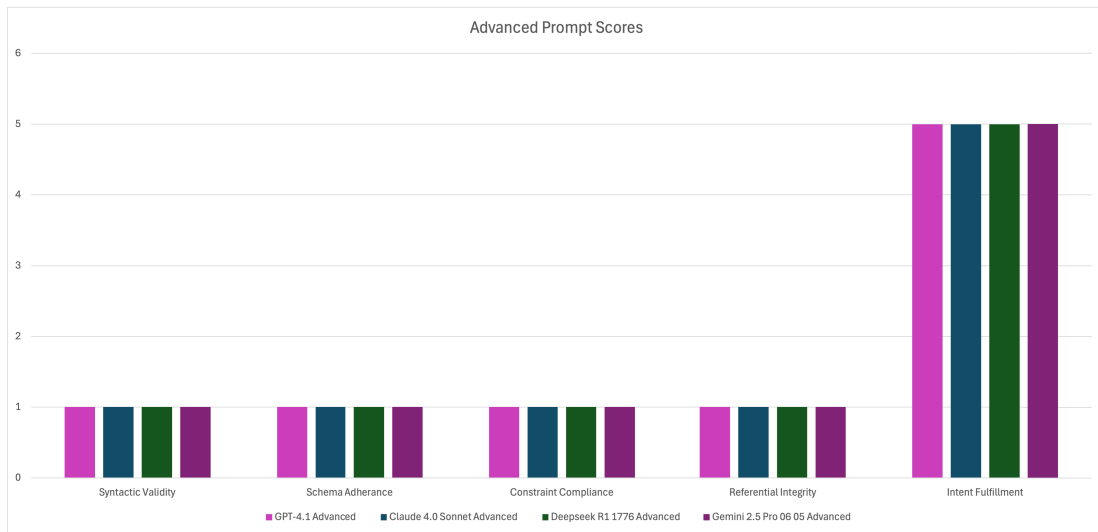


FIGURE 12. Performance of the large language models for the complex advanced prompt.

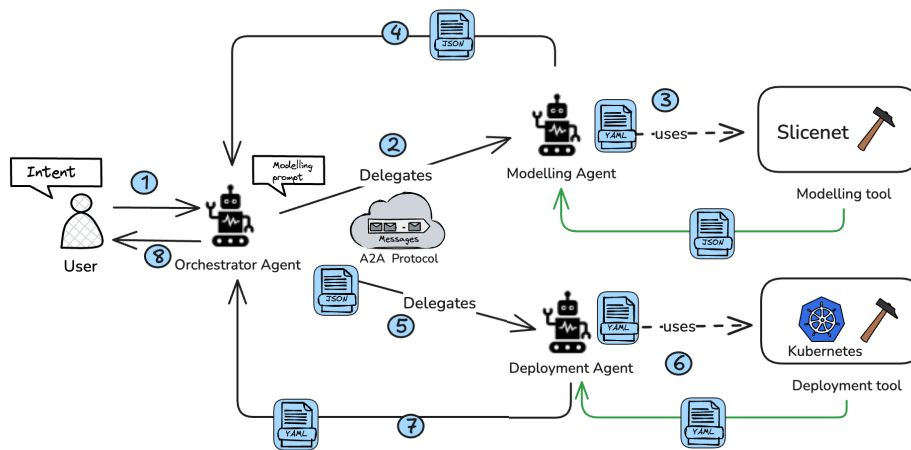


FIGURE 13. Experiment 2's architecture of the multi-agent system for automated network slice lifecycle management. The workflow illustrates the delegation of tasks from the Orchestrator Agent to specialized agents for modeling and deployment.

and compared against human evaluations as a sanity check.

Beyond these four artifact-centric metrics, every YAML configuration produced by an LLM in Experiment 1 is passed through a validation pipeline that checks: (i) syntactic correctness (YAML parsing), (ii) adherence to the Slicenet schema, (iii) referential integrity across all internal lists (network functions, slices, services, slicelets), and (iv) feasibility with respect to the capacity and QoS constraints defined in Section III-A. Configurations that violate any of these checks are rejected and not considered deployable. This pipeline mitigates a broad class of semantic hallucinations where the LLM might otherwise request infeasible resources or generate inconsistent slice definitions.

This experimental design is intentional: before latency, throughput, admission-rate, or resource-utilization KPIs can be benchmarked meaningfully, the generated artifacts must first be deployable, standards-compliant, and feasible with respect to infrastructure constraints. For this reason, Experi-

ment 1 focuses on the artifact-validation stage that precedes downstream controller execution.

Finally, to evaluate a diverse range of frontier LLMs, experiments were conducted using the Perplexity Pro web interface with OpenAI GPT 4.1, Anthropic Claude 4.0 Sonnet, Deepseek R1 1776, Google Gemini 2.5 Pro 06 05 models. As the web interface does not permit manual adjustment of the ‘temperature’ parameter, we implemented a strict protocol to ensure consistency. Each testcase was run 3 times against each model. For each of the ‘N= 3’ runs per test case, a new, isolated chat session was initiated in incognito mode. This ensures that no conversational context from previous interactions could influence the model’s output, thereby keeping all user-controllable variables constant across trials.

The testcase prompts were enclosed in a system prompt that instructed the LLM to translate user intent into a valid YAML file. This system prompt also included a JSON schema that defined the YAML file’s structure and an example to demonstrate the task. Additionally, the “sources to search” feature was disabled, thereby ensuring that the model

does not utilize any external resources, such as web searches, to enhance its generation. This measure was implemented to prevent any external influence from affecting the model's output. A total of 36 YAML files were generated and underwent the scoring process, as outlined previously.

As reported in Figures 10–12, the results indicate that even with limited examples, almost all LLMs generated structurally valid YAML files. In the context of constraint compliance scores, we observed that, with the exception of GPT 4.1, all other models exhibited a failure to adhere to the constraint condition during the generation of the YAML file in instances where an explicit prompt was provided. Additionally, all LLMs demonstrated remarkable proficiency in accurately capturing the user's intent within the YAML files.

Discussion: The benchmark reveals distinct strengths among the evaluated models. GPT 4.1 consistently achieved perfect syntactic validity and the highest schema-adherence and referential-integrity scores across all three prompts. Its outputs always satisfied the negative constraint, and human evaluators rated it highest for intent fulfillment (close to 5 out of 5). Claude 4.0 Sonnet and Deepseek R1 1776, while producing valid YAML and capturing user intent well, occasionally violated constraint conditions when omitting optional fields and thus scored lower on schema adherence. Gemini 2.5 Pro generated syntactically correct YAML but sometimes misordered or incorrectly typed fields, requiring minor post-hoc corrections. Collectively, these findings demonstrate that state-of-the-art LLMs can autonomously handle most slice-modelling tasks even without fine-tuning on networking data; however, careful prompt engineering and rigorous validation remain necessary when deploying models without fine-tuning other than GPT 4.1 in production settings.

We therefore position these results as evidence that LLMs can reliably automate the intent-to-artifact translation stage of slice management. They should not be interpreted as a full operator-grade evaluation of runtime network KPIs under realistic multi-tenant traffic, which would require an integrated testbed with traffic generation, admission control, and closed-loop controllers.

The results of Experiment 1 therefore validate the first AI-driven stage of our workflow: the translation of natural-language intent into a correct and deployable slice description. Experiment 2 complements this by validating the subsequent agent-to-agent orchestration stage, in which the generated slice request is converted into a logical model, a standards-compliant deployment descriptor, and a deployment plan.

B. EXPERIMENT 2: MULTI-AGENT SYSTEM PROOF-OF-CONCEPT FOR NETWORK SLICE MANAGEMENT

To validate the collaborative, intent-driven automation capabilities of our proposed framework, we developed and evaluated a multi-agent system Proof-of-Concept (PoC). The agents in this system are powered by Perplexity Sonar, which is based on the Llama 3.1 405B large language model. The architecture of this system, depicted in Figure 13, is composed of three distinct, specialized agents that collaborate to translate high-level user intent into a fully specified, deployable network slice configuration.

The core components of the architecture are:

- **Orchestrator Agent:** Serves as the primary user interface, responsible for interpreting high-level natural language intent (a user request) and decomposing it into a sequence of structured tasks for other agents.
- **Modelling Agent:** Specializes in network slice design. It receives a detailed technical prompt from the Orchestrator and generates a complete logical model of the network slice, subsequently generating a configuration file compliant with a *Modeling Tool* (e.g., a Slicenet simulator [26]).
- **Deployment Agent:** Translates the validated network model into a concrete, deployable artifact. It receives the model's topological information to generate a standardized Network Service Descriptor (NSD) based on the TOSCA standard and deploy using standard cloud tools like NFV-O or Kubernetes etc.

1) END-TO-END WORKFLOW AND ARTIFACT VALIDATION

A primary contribution of this experiment is to demonstrate not only successful agent-to-agent (A2A) communication but also the generation of a chain of functionally accurate artifacts, each validated by a human expert. The workflow, triggered by a single high-level intent, proceeds as follows:

I need to model & deploy my 6G network infrastructure with 2 slices, 1 for emergency and another for eMBB, such that average utilization ratio is 80% for both.

- 1) **Intent to Technical Prompt:** The Orchestrator Agent first translated the user's intent into a comprehensive, structured technical prompt. This initial artifact detailed the required infrastructure, Network Functions (NFs), resource allocations, and simulation parameters necessary to achieve the user's stated goal.
- 2) **Technical Prompt to Simulation Model:** The Orchestrator delegated this technical prompt to the Modelling Agent. The agent processed it and produced a valid Slicenet configuration YAML file, an artifact directly usable by the simulation tool tested in Experiment 1. This file was saved, and the agent returned a structured JSON object containing the core topology information to the Orchestrator.
- 3) **Simulation Model to Deployment Descriptor:** Finally, the Orchestrator passed the JSON topology object to the Deployment Agent. This agent, in turn, generated a valid TOSCA Network Service Descriptor (NSD) in YAML format, representing the real-world deployment specification for the slice.

2) EXECUTION AND RESULTS

The experiment's execution, completed autonomously in approximately 40 seconds, validated the system's efficiency. The successful generation of the final deployment plan, with steps including `validate_topology`, `allocate_resources`, and `deploy_vnfs`, was confirmed.

Crucially, a domain expert inspected and verified each of the generated artifacts, the technical prompt, Slicenet YAML, and TOSCA NSD. This PoC, therefore, provides two key validations: first, that a multi-agent system can effectively

automate the complex workflow of translating a high-level user intent into a concrete deployment plan; and second, that the intermediate and final artifacts produced by this automated workflow are technically sound and correct. This confirms the viability of our framework for producing reliable, end-to-end automation in network slice management.

Impact and Strength: Our PoC compresses what traditionally requires days or even weeks of human coordination and manual configuration into a fully automated sequence executed in under one minute. By delegating network modelling and descriptor synthesis to specialised agents, the pipeline eliminates human error, ensures that each intermediate artifact (topology specification, simulation model, NSD) is rigorously validated before proceeding, and provides a reproducible foundation for scaling network slicing across multi-domain infrastructures.

For further reference, the complete console output of the multi-agent system proof-of-concept is provided in Appendix B.

The proof-of-concept is intentionally scoped to a single end-to-end workflow in order to verify inter-agent coordination, standards-compliant artifact generation, and execution feasibility. While this does not yet constitute a large-scale multi-request or multi-tenant benchmark, it establishes the missing orchestration layer on top of which broader operator- and provider-led validation campaigns can later be built.

V. CHALLENGES

Integrating Large Language Models (LLMs) into end-to-end network slice management presents several challenges and requires careful consideration:

- 1) **Data Set Availability:** Acquiring relevant and diverse datasets for training and fine-tuning LLMs within a network management context remains a challenge. Simulation techniques can partially address this issue by generating synthetic data, but ensuring the authenticity and representativeness of such data is crucial [26].
- 2) **Computing Power and Hosting:** The computational demands of LLMs, especially large models, can strain existing infrastructure. Advances in developing lighter versions of LLMs such as Phi-1 [31], optimized for deployment on edge devices or cloud environments, are essential for scalability and cost-effectiveness. In addition, as explained in the PoC section, for some classical code/config file generation (e.g., flow rules installation), we may use a local agent with modest infrastructure without fine-tuning or re-training.
- 3) **Network Security Backdoors:** LLMs, like any complex software system, can potentially have vulnerabilities that malicious actors may exploit as backdoors [32]. Utilizing LLMs themselves for continuous analysis and mitigation of such security risks is a proactive approach and an active research area.
- 4) **Trust and Validation:** Ensuring the trustworthiness and reliability of LLM-assisted processes is paramount. Legal and regulatory compliance, such as avoiding SLA violations and network neutrality violation, requires rigorous validation and adherence to standards. Testbeds and Simulation techniques play a crucial role in testing and validating LLM integration

while addressing trust-related concerns. Collaboration with communication regulatory authority and operators is vital for the validation phase before going to commercialization.

- 5) **Interoperability and Standards:** Integrating LLM-assisted functions across heterogeneous network environments requires adherence to interoperability standards. Developing common protocols and interfaces for communication (e.g., northbound interfaces), prompts and data exchange among LLMs and network components is essential for seamless integration.
- 6) **Privacy and Data Protection:** LLMs often operate on sensitive data, raising concerns about privacy and data protection [33]. Implementing robust encryption methods, data anonymization techniques, and access control mechanisms can mitigate privacy risks associated with LLM usage.
- 7) **Hallucinations and Policy Violations:** LLMs may produce syntactically correct yet semantically invalid configurations, for example by requesting resources that exceed infrastructure capacity or by violating latency or reliability constraints. Our current framework mitigates this risk through a multi-stage artifact validation pipeline (Section IV), but integrating constrained decoding, formal verification, and hybrid rule- or optimization-based guards is an important direction for future work, especially in multi-tenant and safety-critical deployments.
- 8) **Operator-grade Evaluation and Baselines:** The present proof-of-concept validates the orchestration/specification layer rather than full network-runtime behavior under large-scale multi-slice traffic. Broader validation under realistic multi-slice, multi-request, and multi-tenant conditions will require coordinated collaboration with telecom, network, and cloud operators/providers, together with access to representative infrastructures, traffic generators, admission-control policies, and controller baselines (e.g., heuristic, optimization-based, and DRL/MARL resource managers).

VI. CONCLUSION

In the era of multi-domain network slicing, the integration of large language models and advanced AI technologies redefines the paradigm of network management. By embracing intelligent, context-aware orchestration, operators can build responsive, secure and efficient infrastructures spanning diverse administrative domains. This research illuminates a path forward in which AI-driven multi-domain NS becomes the cornerstone of next-generation networks, ensuring unparalleled connectivity and user satisfaction.

We proposed a framework that maps LLM-driven agents to the ETSI and 3GPP visions of end-to-end slicing and recent management architectures. The framework delineates how specialized agents collaborate to guide users through intent capture, slice design, deployment, monitoring, and optimization. Our evaluations substantiate the efficacy of this approach. A benchmarking study across four frontier LLMs shows that GPT 4.1 consistently generates syntactically correct YAML slice configurations with near-perfect schema adherence and referential integrity; other models accurately capture user intent but occasionally misapply

explicit constraints, underscoring the need for careful prompt engineering. Complementing this, a multi-agent proof-of-concept demonstrates that a single high-level intent can be autonomously translated into a validated simulation model and deployable TOSCA descriptor in roughly forty seconds, establishing an efficient and reproducible intent-to-descriptor pipeline with minimal manual intervention.

These results should be interpreted within the scope of the paper: we validate the feasibility and utility of an LLM-driven orchestration/specification layer for network slicing through two complementary AI-driven stages, namely intent-to-slice-description generation and multi-agent transformation of that specification into deployable artifacts, not a full-scale network-performance study. Demonstrating KPI gains under realistic multi-slice, multi-request, and multi-tenant conditions remains a broader collaborative next step that depends on engagement with operators, providers, controllers, and experimental infrastructures.

Despite these promising results, several challenges remain. Data scarcity for fine-tuning domain-specific models, the high computational demands of large models, potential security vulnerabilities and backdoors, the need for trust and validation of AI decisions, interoperability across heterogeneous domains, and privacy considerations all pose significant obstacles. Addressing these challenges will require future research into collaborative and federated learning, explainable AI, continuous monitoring and auditing, ethical guidelines, user-centric design, and coordinated efforts among industry, academia and regulators.

VII. FUTURE DIRECTIONS

Our preliminary experiments demonstrate the feasibility of intent-driven network slicing, but fully realizing LLM-enabled management requires advances along several dimensions. Below we outline promising research directions and support each with existing findings from the literature.

- 1) **Collaborative and Federated Learning:** Future systems should enable agents to learn collectively across domains while respecting privacy. Recent work on digital-twin assisted slicing shows that federated multi-agent reinforcement learning, augmented with graph-attention networks, can accurately predict slice demand and reduce communication overhead among distributed controllers [34]. Likewise, federated learning trains models on local data and exchanges only updates; this paradigm preserves tenant privacy yet achieves strong detection accuracy for slice security [35]. Integrating these collaborative methods will improve scalability and resilience of agent-based management.
- 2) **Explainable AI for Network Slicing:** To earn operator trust, LLM decisions must be interpretable. A recent study on network slice recognition emphasizes that explainable AI is essential for making machine-learning models transparent and trustworthy in telecommunications [36]. Incorporating explainability modules into LLM agents will help engineers validate outputs and tune policies.
- 3) **Adaptive Security and Continuous Monitoring:** Embedding security-native capabilities within slicing architectures is another critical direction. Federated learning can support distributed intrusion detection by allowing ML-Agents and Security Agents to share

model updates without exposing raw traffic [35]. Combining this with continuous auditing of agent outputs will help identify backdoors or bias before deployment.

- 4) **Lightweight Models and On-Device Inference:** Large models impose significant computational overhead. Developing compact LLMs or mixture-of-experts architectures that run on edge devices will facilitate on-device intent interpretation and local policy enforcement. Advances in model quantisation and instruction-tuning will be essential to bring LLM capabilities to resource-constrained network elements.
- 5) **Ethics, Privacy and User-Centric Design:** The deployment of LLMs in network management must respect data protection regulations and ethical norms. This involves incorporating privacy-preserving mechanisms, ensuring fairness across tenants, and designing conversational interfaces that give users agency over their services.
- 6) **Standardization and Interoperability:** Finally, ensuring interoperability across administrative domains requires standard interfaces for exchanging intents, policies and models. Aligning agent communication with existing frameworks such as 3GPP, ETSI and GSMA will reduce integration friction and accelerate adoption.

STATEMENTS AND DECLARATIONS

Competing Interests and Funding The authors did not receive support from any organization for the submitted work. In addition, the authors declare that they have no relevant financial or non-financial interests to disclose. Last, open Access funding provided by the Qatar National Library.

AUTHOR CONTRIBUTIONS

AD: Conceptualization, methodology, supervision, Experimental design, manuscript writing and review.

VK: Software implementation, experimentation, validation, manuscript drafting.

MU: Experimental design, data curation, review, editing.

FA: manuscript review.

MD: Theoretical foundation, critical review, editing.

All authors reviewed and approved the final manuscript.

CODE AVAILABILITY

Prompts and proof-of-concept scripts are available at our Github repository [30].

APPENDIX A

SAMPLE LLM AGENTS DEFINITION WITH AGENT ROLE, DESCRIPTION AND ASSOCIATED TOOLS USED BY EACH AGENT

See Listing 1.

APPENDIX B

FULL EXPERIMENT 2 OUTPUT LOG

The following is the complete console output from the execution of the multi-agent system proof-of-concept described in Section IV-B.

Distributed Network Slice Agents Demo

=====

...

WORKFLOW COMPLETED SUCCESSFULLY!

Total Time: 40.37~seconds

Demo completed successfully!

llm-assisted-slice-agents:
 – name : 'On-Device Agent',
 goal : 'Design and Deploy a network slice infrastructure within the constraints given by the network operator',
 prompt : 'You are a helpful on-device assistant...'
 tools :
 – name : 'modeller',
 agent : 'Slice modelling Agent',
 description : 'Use this tool to perform simulation of different network scenarios',
 input : 'network slice modelling requirements',
 output : 'slice topology recommendations'
 – name : 'deployer',
 agent : 'Slice Deployment Agent',
 description : 'Use this tool to perform deployment of a specific network topology',
 input : 'network slice deployment information',
 output : 'deployed network slice information'

LISTING 1. LLM Assisted Slice Agents Definitions in Yaml template.

ACKNOWLEDGMENT

QNL in Qatar has funded the fees for the Open Access publication.

REFERENCES

- [1] C. Jacquenet, "Optimized, automated, and protective: An Operator's view on future networks," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1350–1359, Jun. 2021.
- [2] M. Chahbar, G. Diaz, A. Dandoush, C. Cerin, and K. Ghomid, "A comprehensive survey on the E2E 5G network slicing model," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 49–62, Mar. 2021.
- [3] S. Wijethilaka and M. Liyanage, "Survey on network slicing for Internet of Things realization in 5G networks," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 957–994, 2nd Quart., 2021.
- [4] R. Raman, R. Singh, Z. Gupta, S. Verma, A. Rajput, and S. M. Parikh, "Wireless communication with extreme reliability and low latency: Tail, risk and scale," in *Proc. 5th Int. Conf. Contemp. Comput. Informat. (IC3I)*, Dec. 2022, pp. 1699–1703.
- [5] K. Mehmood, K. Kralevska, and D. Palma, "Intent-driven autonomous network and service management in future cellular networks: A structured literature review," *Comput. Netw.*, vol. 220, Jan. 2023, Art. no. 109477.
- [6] A. Dandoush, A. Kammoun, g. diaz, n. achir, and N. Tabbane, "Enabling new generation services with multi-level delegation architecture of slicing technology," in *Innovation and Technological Advances for Sustainability*, 1st ed. Boca Raton, FL, USA: CRC Press, 2024, p. 11, doi: 10.1201/9781003496724.
- [7] Next Generation Protocols (NGP); E2E Network Slicing Reference Framework and Information Model, Standard ETSI GR NGP 011, 2018.
- [8] Network Functions Virtualisation (NFV); Ecosystem; Report on SDN Usage in NFV Architectural Framework, Standard ETSI GS NFV-EVE 005, Dec. 2015.
- [9] Network Functions Virtualisation (NFV); Management and Orchestration, Standard GS NFV-MAN 001, Dec. 2014.
- [10] (Mar. 2021). *TMF641 Service Ordering API User Guide V4.1.1*. [Online]. Available: <https://www.tmfforum.org/>
- [11] Ethernet Ordering Technical Specification, Standard J-SPEC-001/MEF 57, Jul. 2017. [Online]. Available: <https://www.mef.net/wp-content/uploads/2017/07/MEF-57.pdf>
- [12] L. Bariyah, Q. Zhao, H. Zou, Y. Tian, F. Bader, and M. Debbah, "Large language models for telecom: The next big thing?," *Comm. Mag.*, vol. 62, pp. 84–90, Nov. 2024. [Online]. Available: <https://doi.org/10.1109/MCOM.001.2300364>
- [13] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?," *Proc. ACM Netw.*, vol. 2, no. 2, pp. 1–25, Jun. 2024.
- [14] O. G. Lira, O. M. Caicedo, and N. L. S. da Fonseca, "Large language models for zero touch network configuration management," *IEEE Commun. Mag.*, vol. 63, no. 7, pp. 146–153, Jul. 2025.
- [15] N. Van Tu, J.-H. Yoo, and J. W.-K. Hong, "Towards intent-based configuration for network function virtualization using in-context learning in large language models," in *Proc. IEEE Netw. Oper. Manage. Symp.*, May 2024, pp. 1–8.
- [16] X. Zhou, R. Li, T. Chen, and H. Zhang, "Network slicing as a service: Enabling enterprises' own software-defined cellular networks," *IEEE Commun. Mag.*, vol. 54, no. 7, pp. 146–153, Jul. 2016.
- [17] J. Hong, N. V. Tu, and J. W.-K. Hong, "A comprehensive survey on LLM-based network management and operations," *Int. J. Netw. Manage.*, vol. 35, no. 6, p. 70029, Nov. 2025.
- [18] F. Liu, B. Farkiani, and P. Crowley, "Large language models for computer networking operations and management: A survey on applications, key techniques, and opportunities," *Comput. Netw.*, vol. 271, 2025, Art. no. 111614. [Online]. Available: <https://doi.org/10.1016/j.comnet.2025.111614>
- [19] J. McNamara et al., "NLP powered intent based network management for private 5G networks," *IEEE Access*, vol. 11, pp. 36642–36657, 2023.
- [20] S. Tarkoma, R. Morabito, and J. Sauvola, "AI-native interconnect framework for integration of large language model technologies in 6G systems," 2023, *arXiv:2311.05842*.
- [21] K. Mao, "Multi-agent reinforcement learning driven resource game optimization for network slicing in MEC-enabled HetNets," *Sci. Rep.*, vol. 16, p. 3231, 2026. [Online]. Available: <https://doi.org/10.1038/s41598-025-33190-5>
- [22] J. Tong et al., "WirelessAgent: Large language model agents for intelligent wireless networks," 2025, *arXiv:2505.01074*.
- [23] *AI and ML-Enablers for Beyond 5G Networks*, 5G PPP Technology Board, p. 141, 2021, doi: 10.5281/zenodo.4299895. [Online]. Available: <https://5g-ppp.eu/wp-content/uploads/2021/05/AIMLforNetworks-v1-0.pdf>
- [24] A. Vaswani et al., "Attention is all you need," 2017, *arXiv:1706.03762*.
- [25] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, Jun. 2019, pp. 4171–4186.
- [26] V. KumarSkandPriya, A. Dandoush, and G. Diaz, "Slicenet: A simple and scalable flow-level simulator for network slice provisioning and management," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, Oct. 2023, pp. 1–6.
- [27] *Management and Orchestration of 5G Networks; Network Resource Model (NRM); Stage 2 and Stage 3*, document TS 28.541, 3GPP, 2023.
- [28] *Network Functions Virtualisation (NFV) Release 3; Protocols and Data Models; Network Service Descriptor File Structure Specification*, ETSI, Sophia Antipolis Cedex, France, Group Specification ETSI GS NFV-SOL 007, 2021.
- [29] Network Functions Virtualisation (NFV) Release 4; Protocols and Data Models; NFV Descriptors Based on Tosca Specification, Standard ETSI GS NFV-SOL 001, 2022.
- [30] A. Dandoush, V. Kumarskandpriya, M. Uddin, and U. Khalil. (2025). *The Prompts Used in the Paper's Experiment 1*. [Online]. Available: <https://github.com/dandoush/Multi-Agent-LLMs-prompts/edit/main/README.md>
- [31] S. Gunasekar et al., "Textbooks are all you need," 2023, *arXiv:2306.11644*.
- [32] H. Yang, K. Xiang, M. Ge, H. Li, R. Lu, and S. Yu, "A comprehensive overview of backdoor attacks in large language models within communication networks," *IEEE Netw.*, vol. 38, no. 6, pp. 211–218, Nov. 2024.
- [33] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-BASED agentic systems," presented at the Netw. Distrib. Syst. Secur. Symp., Feb. 2025, doi: 10.14722/ndss.2025.241131.
- [34] D. Ayepah-Mensah, G. Sun, Y. Pang, and W. Jiang, "Adaptive digital twin and communication-efficient federated learning network slicing for 5G-enabled Internet of Things," 2024, *arXiv:2407.10987*.
- [35] R. Moreira et al., "An intelligent native network slicing security architecture empowered by federated learning," 2024, *arXiv:2410.05312*.
- [36] D. Ray, "Network slice recognition with explainable machine learning," *J. Adv. Artif. Intell.*, vol. 1, no. 7, pp. 9–14, 2025.